

Laboratory Work #2: Normalization and DDL

Task 1: Normalization

1.1 Normal Form Violations Analysis

The initial table **Booking_receipt** contains several design flaws that violate the standard normal forms. For the purpose of this analysis, we assume the table includes: `receipt_id`, `passenger_names`, `flight_id`, `departure_airport`, `arrival_airport`, and `price`.

- **1NF Violation:** The `passenger_names` column contains multiple values in a single cell (e.g., "John Doe, Jane Doe") when a group booking is made. This violates atomicity.
- **2NF Violation:** Assuming a composite primary key of (`receipt_id`, `flight_id`), the `price` attribute might depend only on the `flight_id` rather than the entire composite key. This creates a partial functional dependency.
- **3NF Violation:** The `departure_airport` depends on the `flight_id`. Since `flight_id` is a non-key attribute relative to the `receipt_id` (the primary identifier of a single receipt), this is a transitive dependency.

1.2 Step-by-Step Normalization to 3NF

First Normal Form (1NF)

To achieve 1NF, we ensure all attributes are atomic. We split the multi-valued passenger names into separate rows.

- **Table: Booking_Receipt_1NF**
- **Attributes:** `receipt_id`, `passenger_name`, `flight_id`, `departure_airport`, `arrival_airport`, `price`.

Second Normal Form (2NF)

To achieve 2NF, we remove partial dependencies. We separate the receipt details from the flight-specific details.

- **Table: Receipts**
 - Attributes: receipt_id, passenger_name, flight_id (FK)
- **Table: Flight_Details**
 - Attributes: flight_id, departure_airport, arrival_airport, price

Third Normal Form (3NF)

To achieve 3NF, we remove transitive dependencies. In the **Flight_Details** table, if the airport names depend on a specific airport code, those should be moved to their own table.

- Table: Receipts
 - Attributes: receipt_id, passenger_name, flight_id (FK)
- Table: Flights
 - Attributes: flight_id, dep_airport_code (FK), arr_airport_code (FK), price
- Table: Airports
 - Attributes: airport_code, airport_name

1.3 Functional Dependencies (FDs)

The following functional dependencies were identified during the normalization process:

- **FD1:** receipt_id → passenger_name, flight_id
- **FD2:** flight_id → dep_airport_code, arr_airport_code, price
- **FD3:** airport_code → airport_name

1.4 Flights Table 3NF Analysis

The **Flights** table is in 3NF because:

1. It is in 2NF (all non-prime attributes are fully functionally dependent on the primary key **flight_id**).

2. There are no transitive dependencies. Attributes like **price** and airport codes depend directly on the **flight_id**. The actual airport names are moved to a separate **Airports** table to ensure that a change in an airport's name doesn't require updates in every flight row.

1.5 Task 2 3NF Tables Implementation

The following SQL implements the normalized structure for Task 2.-- Creating the Flights table

```
CREATE TABLE Flights (  
  
    flight_id INT PRIMARY KEY,  
  
    origin_city VARCHAR(100) NOT NULL,  
  
    destination_city VARCHAR(100) NOT NULL,  
  
    base_price DECIMAL(10, 2) NOT NULL  
  
);  
  
-- Inserting 5 rows of data  
  
INSERT INTO Flights (flight_id, origin_city, destination_city, base_price) VALUES  
  
(101, 'Almaty', 'Astana', 25000.00),  
  
(102, 'Astana', 'London', 150000.00),  
  
(103, 'Almaty', 'Dubai', 85000.00),  
  
(104, 'Shymkent', 'Almaty', 15000.00),  
  
(105, 'Astana', 'Paris', 180000.00);
```

Task 2: DDL Section

2.1 Table Creation with Constraints

We define the initial structure for the `Airline_info` table, ensuring data integrity through `NOT NULL` and `PRIMARY KEY` constraints. `CREATE TABLE Airline_info (`

```
    airline_id SERIAL PRIMARY KEY,  
  
    airline_name VARCHAR(255) NOT NULL,  
  
    icao_code CHAR(3) NOT NULL UNIQUE,  
  
    price DECIMAL(10, 2) NOT NULL,  
  
    departing_gate INT,  
  
    info TEXT  
  
);
```

2.2 Schema Alterations

Following the requirements, we perform the following modifications to the schema:-- 1. Rename `Airline_info` to `Airline`

```
ALTER TABLE Airline_info RENAME TO Airline;
```

-- 2. Rename `price` to `ticket_price`

```
ALTER TABLE Airline RENAME COLUMN price TO ticket_price;
```

-- 3. Change `departing_gate` type from `INT` to `TEXT`

```
ALTER TABLE Airline ALTER COLUMN departing_gate TYPE TEXT;
```

-- 4. Drop the `info` column

ALTER TABLE Airline DROP COLUMN info;

2.3 Foreign Key Relationships Constraints

To maintain referential integrity between the booking system and the airlines, we establish foreign key relationships.-- Creating a secondary table to demonstrate FK relationship

```
CREATE TABLE Tickets (  
  
    ticket_id SERIAL PRIMARY KEY,  
  
    passenger_name VARCHAR(100) NOT NULL,  
  
    airline_id INT NOT NULL,  
  
    flight_date DATE NOT NULL,  
  
    CONSTRAINT fk_airline  
  
        FOREIGN KEY(airline_id)  
  
        REFERENCES Airline(airline_id)  
  
    ON DELETE CASCADE  
  
);
```

Table Name	Column Name	Constraint Type
Airline ▾	airline_id ▾	PRIMARY KEY ▾
Airline ▾	airline_name ▾	NOT NULL ▾
Tickets ▾	ticket_id ▾	PRIMARY KEY ▾
Tickets ▾	airline_id ▾	FOREIGN KEY ▾

Table Name	Column Name	Constraint Type
Tickets ▾	passenger_name ▾	NOT NULL ▾

The document concludes the laboratory work regarding the normalization of database structures and the application of Data Definition Language to manage relational schemas.